SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

| | | | |
|---|---|---|---|
| Course Code: | CSA1716 | Course Title: | Artificial Intelligence |
| CO Assessed: | CO1 – Apply (BL3) | Assessment: | Assessment Tool 2 – Dry Run Evaluation |
| Weightage: | 35% of CIA | Duration / Marks: | 60 mins / 100 marks |
| CO1 Statement: | Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) | Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) | Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) |
| Student Name: | M. Srilekha | Reg. No.: | 1919260007 |
| Section / Batch: | 2026 | Date: | 10/09/2026 |

Code of Conduct
I M. Srilekha 1919260007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: M. Srilekha

Instructions
Read each question carefully; answer all four questions (25 marks each, 100 in total).
Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

| Section / Topic | Focus | Question Nos. |
|---|---|---|
| 1. Foundations and History of AI | Four approaches to AI, Turing Test, eras and milestones | Q2 |
| 2. Intelligent Agents: Agents and Environments | Agent, percepts, actions, agent function and program, PEAS | Q1 |
| 3. The Concept of Rationality | Performance measure, percept sequence, rational vs omniscient | Q1 |
| 4. The Nature of Environments | Task-environment properties and their classification | Q1 |
| 5. The Structure of Agents | Simple reflex and model-based reflex agent programs | Q1 |
| 6. Problem-Solving Agents | Goal formulation, problem formulation, search, execution | Q2 |
| 7. Problem Formulation | States, initial state, actions, goal test, path cost; belief states | Q2, Q4 |
| 8. Searching for Solutions | Search graph, frontier, explored set, node expansion | Q3 |
| 9. Uninformed Search Strategies | Dry runs of BFS, DFS and uniform-cost search | Q3 |
| 10. Search with Partial Information | Sensorless problems, belief states, percept update | Q4 |

## Annexure A – Dry Run Evaluation

## Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and –1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent.

Answer: PEAS — Performance: +10 for each square cleaned and –1 for each movement. Environment: two squares A and B, each clean or dirty; dirt does not reappear. Actuators: Suck, Left, Right, NoOp. Sensors: perceive current location and the status of the current square.

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each.

Answer: Observability: partially observable, because only the current square is sensed. Agents: single-agent, because only the vacuum acts. Determinism: deterministic, because actions have predictable outcomes. Episodic/sequential: sequential, because actions affect later states. Static/dynamic: static, because dirt does not reappear and the world does not change independently. Discrete/continuous: discrete, because locations, dirt status, actions and time steps are discrete.

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step).

| Step | X: Percept | X: Action | Y: Percept | Y: Action | Y: Known-Clean Squares |
|---|---|---|---|---|---|
| 1 | [A, Dirty] | Suck | [A, Dirty] | Suck | {A} |
| 2 | [A, Clean] | Right | [A, Clean] | Right | {A} |
| 3 | [B, Dirty] | Suck | [B, Dirty] | Suck | {A, B} |
| 4 | [B, Clean] | Left | [B, Clean] | NoOp | {A, B} |
| 5 | [A, Clean] | Right | [B, Clean] | NoOp | {A, B} |
| 6 | [B, Clean] | Left | [B, Clean] | NoOp | {A, B} |

(d) Compute the total performance of X and of Y over the 6 steps.

Answer: Agent X cleans A and B (+20) and moves four times (–4), so its score is 20 − 4 = 16. Agent Y cleans A and B (+20) and moves once (–1), then uses NoOp, so its score is 20 − 1 = 19.

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent.

Answer: Agent Y is more rational for this task because it remembers that both squares are clean and avoids unnecessary movement. At step 4, both perceive [B, Clean]. X uses only the current percept and moves Left; Y uses its internal state {A, B} and chooses NoOp. A rational agent selects actions that maximize expected performance using available percepts and knowledge. An omniscient agent would know the actual outcome of every action in advance; a rational agent does not need such perfect knowledge.

## Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

Answer: The four classifications are filled into the table below. The Turing Test concerns acting humanly; Aristotle's logic concerns thinking rationally; the rational-agent definition concerns acting rationally.

(a) Complete the table

Answer: See the completed classification table below.

| System / Idea | Approach to AI | Era / Milestone |
|---|---|---|
| Turing Test proposed (1950) | Acting humanly | 1950; behavioural approach / Turing Test |
| General Problem Solver (Newell and Simon, 1957) | Thinking humanly | 1950s; cognitive simulation / symbolic problem solving |
| Aristotle's syllogisms and formal logic | Thinking rationally | Ancient Greece; foundations of formal logic |
| An agent that does the right thing to maximise expected performance | Acting rationally | Modern rational-agent approach; AI decision theory |

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost.

Answer: State: (x, y), where x is the amount in jug X (0–4) and y is the amount in jug Y (0–3). Initial state: (0, 0). Actions: fill either jug, empty either jug, and pour from one jug into the other until the source is empty or the target is full. Transition model: apply the chosen action and obey jug capacities. Goal test: x = 2. Path cost: 1 per action, so the solution with the fewest actions has the lowest cost.

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged).

Answer: From (0, 0): fill X → (4, 0); fill Y → (0, 3). From (4, 0): empty X → (0, 0); fill Y → (4, 3); pour X into Y → (1, 3). Other actions leave the state unchanged and are omitted.

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost.

Answer: There are 5 possible amounts in X (0–4) and 4 possible amounts in Y (0–3), giving 5 × 4 = 20 states. One shortest solution is (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0). The goal x = 2 is reached in 6 actions, so the path cost is 6.

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts.

Answer: Goal formulation: obtain exactly 2 litres in jug X. Problem formulation: define states, legal jug operations, transitions, goal test and unit action cost. Search: find the lowest-cost sequence of actions from (0,0) to a state with x = 2. Execution: perform each action in order. The agent can execute the plan without new percepts because the jug capacities and action outcomes are known and deterministic, assuming no unexpected changes.

## Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

| Edge | Cost | Edge | Cost |
|---|---|---|---|
| S – A | 1 | B – D | 1 |
| S – B | 4 | C – G | 6 |
| A – C | 2 | D – G | 2 |
| A – D | 5 | | |

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue.

Answer: BFS explores level by level using a FIFO queue. The goal is removed at iteration 6. The path is S → A → C → G, with cost 1 + 2 + 6 = 9.

| Iteration | Node Dequeued | Queue after Expansion | Explored Set |
|---|---|---|---|
| 1 | S | [A, B] | {S} |
| 2 | A | [B, C, D] | {S, A} |
| 3 | B | [C, D] | {S, A, B} |
| 4 | C | [D, G] | {S, A, B, C} |
| 5 | D | [G] | {S, A, B, C, D} |
| 6 | G | [] (goal) | {S, A, B, C, D} |

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left.

Answer: See the completed stack trace below; the stack top is shown on the left.

Answer: DFS follows the first alphabetically ordered branch deeply. It reaches G along S → A → C → G after removing 4 nodes. Its path cost is 9.

| Iteration | Node Popped | Stack after Pushing Children |
|---|---|---|
| 1 | S | [A, B] |
| 2 | A | [C, D, B] |
| 3 | C | [G, D, B] |
| 4 | G | [D, B] (goal) |

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found.

Answer: See the completed priority-queue trace below.

Answer: UCS always removes the frontier node with the lowest cumulative path cost. When B is expanded, the route to D improves from cost 6 to cost 5. Expanding D improves the route to G from 9 to 7. UCS therefore returns S → B → D → G with total cost 4 + 1 + 2 = 7.

| Iteration | Node Expanded | g(n) | Priority Queue after Expansion |
|---|---|---|---|
| 1 | S | 0 | [A:1(S), B:4(S)] |
| 2 | A | 1 | [C:3(A), B:4(S), D:6(A)] |

| Iteration | Node Expanded | g(n) | Priority Queue after Expansion |
|---|---|---|---|
| 3 | C | 3 | [B:4(S), D:6(A), G:9(C)] |
| 4 | B | 4 | [D:5(B), G:9(C)] |
| 5 | D | 5 | [G:7(D)] (replace G:9) |
| 6 | G | 7 | [] (goal) |

(d) Complete the comparison table.

Answer: BFS and UCS each remove 6 nodes here, but BFS ignores edge weights while UCS prioritizes cumulative cost. DFS removes only 4 nodes in this graph, but DFS is not generally optimal and can return an expensive path.

| Strategy | Path Found | Path Cost | Nodes Removed from Frontier | Optimal Here? |
|---|---|---|---|---|
| BFS | S–A–C–G | 9 | 6 | No (not least cost) |
| DFS | S–A–C–G | 9 | 4 | No guarantee |
| UCS | S–B–D–G | 7 | 6 | Yes |

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example).

Answer: UCS finds a cheaper route because it expands nodes by cumulative path cost, not depth. DFS happens to reach the goal quickly because the first branch leads to G, but this depends on the graph and ordering; in general it is not guaranteed to find a shortest or least-cost path. The explored set prevents repeated expansion and cycles. For example, at BFS iteration 3, B has neighbour D, which is already in the queue, so D must not be added again.

## Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

| State | Description | State | Description |
|---|---|---|---|
| S1 | (L, D, D) | S5 | (L, C, D) |
| S2 | (R, D, D) | S6 | (R, C, D) |
| S3 | (L, D, C) | S7 | (L, C, C) |
| S4 | (R, D, C) | S8 | (R, C, C) |

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost.

Answer: Initial belief state: {S1, S2, S3, S4, S5, S6, S7, S8}. Actions: Left, Right and Suck, applied to every possible physical state in the belief set. Goal test: every state in the belief set has both squares clean (only S7 and S8 qualify). Path cost: 1 per action.

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action.

Answer: The belief-state sequence is completed in the table below.

| Step | Action | Belief State after Action |
|---|---|---|
| 0 | (start) | {S1, S2, S3, S4, S5, S6, S7, S8} |
| 1 | Right | {S2, S4, S6, S8} |
| 2 | Suck | {S4, S6, S8} |
| 3 | Left | {S3, S5, S7} |
| 4 | Suck | {S5, S7} |

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update).

Answer: The predicted and updated belief states are completed in the table below.

| Step | Action | After Action (Predicted) | Percept | After Percept (Updated) |
|---|---|---|---|---|
| 0 | (start) | {S1, S2, S3, S4, S5, S6, S7, S8} | [Left, Dirty] | {S1, S3} |
| 1 | Suck | {S5, S7} | [Left, Clean] | {S5, S7} |
| 2 | Right | {S6, S8} | [Right, Dirty] | {S6} |

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states?

Answer: With 8 physical states, the number of possible belief states is 2■ = 256, because each physical state may either be included or excluded. This is harder than physical-state search because the search space can grow exponentially.

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational.

Answer: Each percept removes states inconsistent with the observed location and dirt status. In part (c), [Left, Dirty] reduces the initial set to {S1, S3}; after Suck, the predicted set is {S5, S7}, and [Left, Clean] keeps those states. After Right, the prediction is {S6, S8}; [Right, Dirty] leaves only {S6}. In the sensorless case, no percepts are available, so uncertainty remains across multiple states. The sensorless plan is rational because it chooses actions that make progress toward cleaning under the available information, even without knowing the exact physical state.

Rubrics for Calculation

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| Understanding of Search Problem | 20 | Clearly understands the search problem, initial state, goal state, and search strategy. | Understands the problem with minor misunderstandings. | Demonstrates partial understanding of the search problem. | Shows little or no understanding of the search problem. |
| State Expansion and Dry Run Execution | 30 | Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors. | Performs most iterations correctly with minor mistakes in state expansion or data structure updates. | Performs only some iterations correctly; several errors in tracing or node expansion. | Unable to correctly perform the dry run or expand states. |
| Application of Search Algorithm | 20 | Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules. | Applies the algorithm correctly with a few procedural errors. | Applies only basic algorithm steps with noticeable mistakes. | Fails to apply the correct search algorithm. |
| Accuracy of Solution Path | 20 | Identifies the correct traversal order/solution path and goal state with proper justification. | Solution path is mostly correct with minor errors. | Partially correct solution path; goal may not be reached correctly. | Incorrect or incomplete solution path. |
| Accuracy of Solution Path | 10 | Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented. | Presentation is clear with minor organizational issues. | Limited explanation and organization; reasoning is partially clear. | Poor presentation with unclear or missing reasoning. |

| Q. No. / Criteria | Understanding of Search Problem | State Expansion and Dry Run Execution | Application of Search Algorithm | Accuracy of Solution Path | Accuracy of Solution Path | Sub. Total (20) | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |
| 4 | | | | | | | |
| Total | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |